

The background image is a dark, atmospheric scene of a city in ruins. Tall, skeletal buildings with missing windows and crumbling facades line a street. Several cars are visible, some appearing damaged or abandoned. Streetlights are positioned along the road. The overall tone is somber and desolate, with a heavy sky. The text is overlaid in the center of the image.

Unreal Engine 포트폴리오

SCAVENGER'S EDGE

김정환



목차

1. 게임 소개
2. 프로젝트 구조
3. 핵심기능
4. 트러블슈팅
5. 보완점 및 반성점



SCAVENGER'S EDGE

장르 : 액션 RPG

작업기간 : 65일 (26.01.14~26.03.20)

개발인원 : 1인

툴 및 언어 : Visual Studio C++, Unreal Engine 5

먼 미래 모종의 이유로 모든 문명이 멸망한 포스트 아포칼립스 세계관에서 위험한 폐허를 떠돌며 자원을 수집해나가는 액션 RPG 게임입니다.

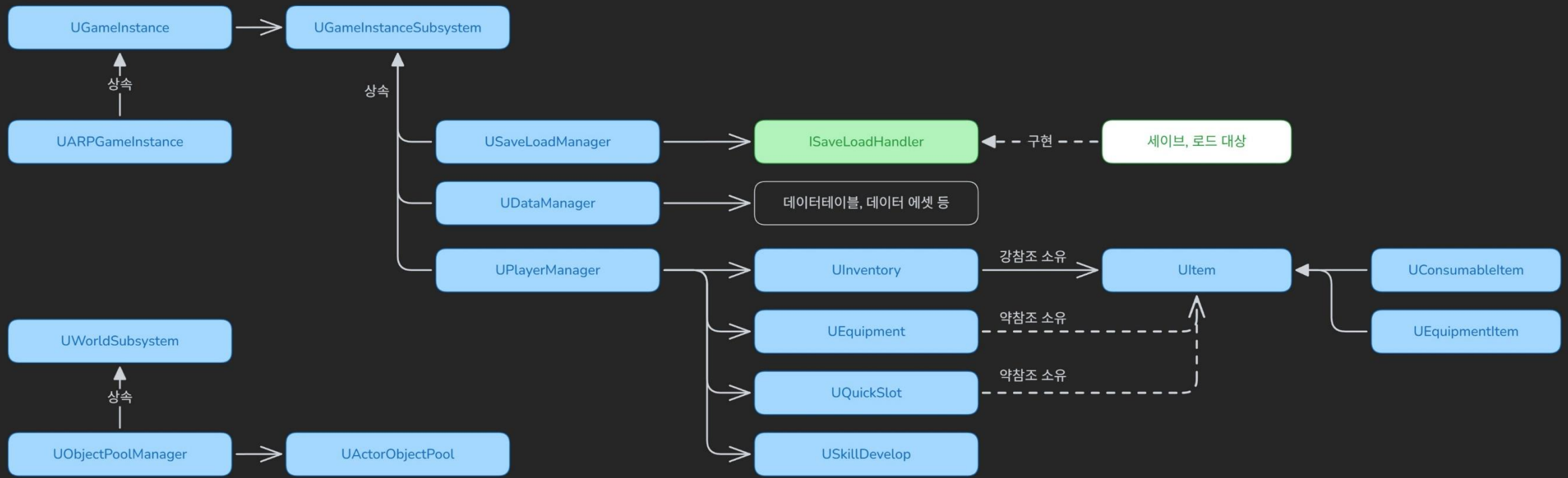
빈딕투스 : 디파인 페이트를 레퍼런스로 프로젝트를 진행하며

콤보를 통한 공격과 스킬 사용을 중점으로 작업했습니다.

플레이어가 그저 콤보를 난타하는 것이 아니라 상황에 따라 전략적으로 액션을 취할 수 있도록 캐릭터의 액션을 통한 버프, 디버프 부여 기능과 스킬 시스템을 구현했습니다.

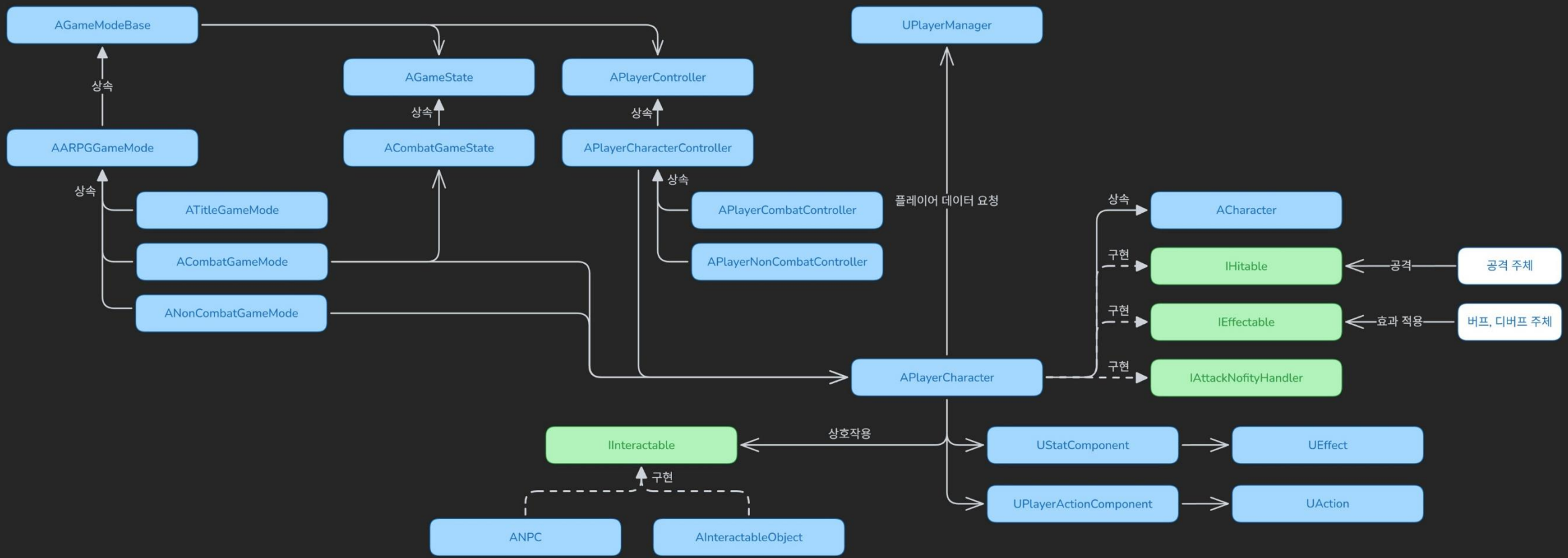
주요 클래스

게임에서 전역적으로 사용하는 매니저 클래스들의 구성입니다.



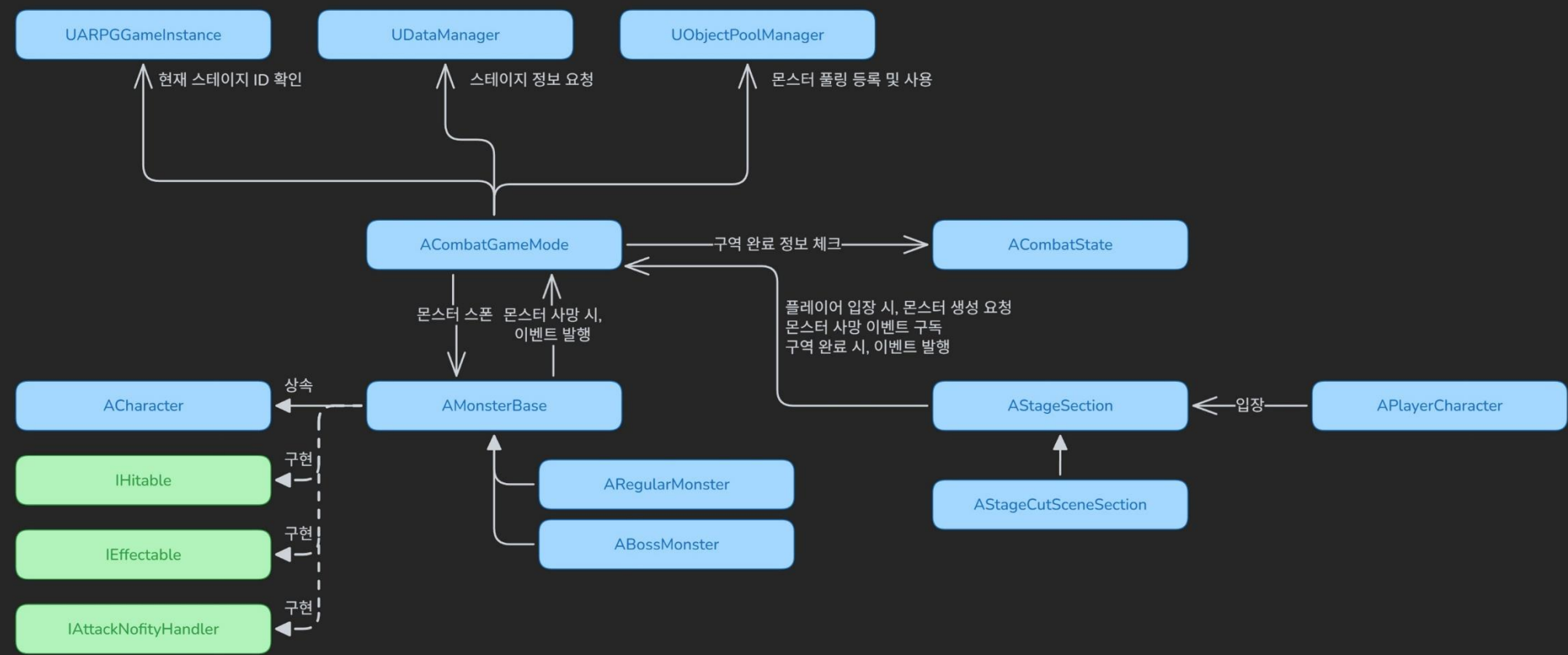
주요 클래스

플레이어의 게임 진행에 관련된 클래스의 구성입니다.



주요 클래스

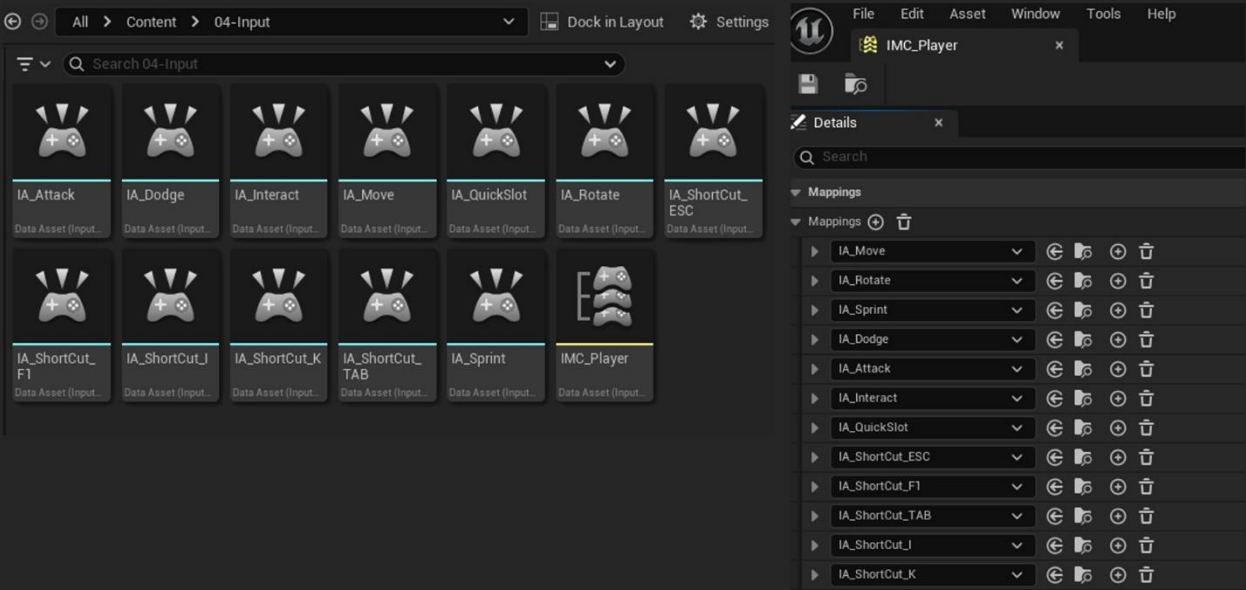
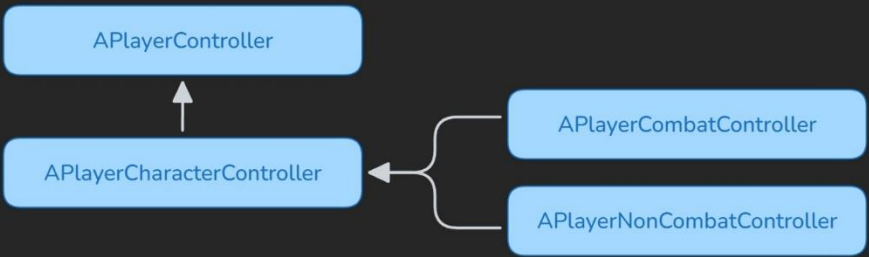
전투 진행에 관련된 클래스들의 구성입니다.



플레이어 조작

PlayerController

Enhanced Input System을 이용하여
캐릭터의 이동, 회피, 상호작용, 공격, 단축키 입력 기능을 구현했습니다.
PlayerController를 세분화하여 레벨의 성격에 따라 필요한 입력을 처리할 수 있도록 작업했습니다.



```
void APlayerCharacterController::SetupInputComponent()
{
    Super::SetupInputComponent();

    if (UEnhancedInputComponent* InputComp = Cast<UEnhancedInputComponent>(InputComponent))
    {
        InputComp->BindAction(MoveAction, ETriggerEvent::Triggered, this, &APlayerCharacterController::InputMove);
        InputComp->BindAction(MoveAction, ETriggerEvent::Completed, this, &APlayerCharacterController::InputMoveEnd);

        InputComp->BindAction(RotateAction, ETriggerEvent::Triggered, this, &APlayerCharacterController::InputRotate);

        InputComp->BindAction(SprintAction, ETriggerEvent::Started, this, &APlayerCharacterController::InputSprintStart);
        InputComp->BindAction(SprintAction, ETriggerEvent::Completed, this, &APlayerCharacterController::InputSprintEnd);

        InputComp->BindAction(InteractAction, ETriggerEvent::Triggered, this, &APlayerCharacterController::InputInteract);

        InputComp->BindAction(ShortCutAction_ESC, ETriggerEvent::Started, this, &APlayerCharacterController::InputShortCutESC);
        InputComp->BindAction(ShortCutAction_Tab, ETriggerEvent::Started, this, &APlayerCharacterController::InputShortCutTAB);
        InputComp->BindAction(ShortCutAction_F1, ETriggerEvent::Started, this, &APlayerCharacterController::InputShortCutF1);
        InputComp->BindAction(ShortCutAction_Key_I, ETriggerEvent::Started, this, &APlayerCharacterController::InputShortCutI);
        InputComp->BindAction(ShortCutAction_Key_K, ETriggerEvent::Started, this, &APlayerCharacterController::InputShortCutK);
    }
}
```

```
void APlayerCombatController::SetupInputComponent()
{
    Super::SetupInputComponent();

    if (UEnhancedInputComponent* InputComp = Cast<UEnhancedInputComponent>(InputComponent))
    {
        InputComp->BindAction(DodgeAction, ETriggerEvent::Triggered, this, &APlayerCombatController::InputDodge);

        InputComp->BindAction(AttackAction, ETriggerEvent::Triggered, this, &APlayerCombatController::InputAttack);
        InputComp->BindAction(AttackAction, ETriggerEvent::Completed, this, &APlayerCombatController::InputAttackEnd);

        InputComp->BindAction(QuickSlotAction, ETriggerEvent::Triggered, this, &APlayerCombatController::InputQuickSlot);
    }
}
```

핵심 기능 1. 플레이어

플레이어 기능 : 스탯과 효과

UStatComponent

캐릭터의 체력, 스테미너, 공격력 같은 스탯을 관리하는 액터 컴포넌트입니다.
캐릭터가 가진 기본 스탯뿐만 아니라 장비와 효과로 얻은 스탯도 반영할 수 있습니다.

체력, 스테미너, 스킬 게이지는 자원 요소로 처리합니다.
자원 요소들은 값 변동과 고갈 시, 이벤트를 발행하여 UI 연동하는 등의 용도로 사용했습니다.

자원 중에서 스테미너와 스킬 게이지는 캐릭터의 액션과 함께 사용됩니다.
모든 액션은 스테미너를 소모하여 작동하고 스테미너는 액션 중이지 않다면 천천히 회복됩니다.
액션 중 스테미너가 0이 되면 5초간 천천히 걸으며 스테미너가 회복되지 않는 페널티 기능을 구현했습니다.

스킬 게이지는 전투 중 획득 가능하고 스킬 게이지를 소모하여 액션에 부여된 효과를 발동합니다.

Lv. 5100 / 500

스태이터스

Health	305 + (45) = 350
Stamina	105 + (5) = 110
Skill	100 + (0) = 100
Attack	35 + (15) = 50
Defense	45 + (45) = 90
Critical Percent	10 + (15) = 25
Critical Damage	100 + (25) = 125



```
void UStatComponent::Init(const TMap<ECharacterStatType, uint32>& _initStat)
{
    Stat = _initStat;
    GetResource(ECharacterResourceType::HEALTH).Init(GetStat(ECharacterStatType::HEALTH));
    GetResource(ECharacterResourceType::STAMINA).Init(GetStat(ECharacterStatType::STAMINA));
    GetResource(ECharacterResourceType::SKILL).Init(GetStat(ECharacterStatType::SKILL), false);
}

void UStatComponent::Init(const TMap<ECharacterStatType, uint32>& _initStat,
                          const TMap<ECharacterStatType, uint32>& _equipmentStat)
{
    EquipmentStat = _equipmentStat;
    Init(_initStat);
}

uint32 UStatComponent::GetStat(
    ECharacterStatType _type,
    bool _bExceptEffect) const
{
    uint32 Result = Stat[_type];

    Result += EquipmentStat[_type];

    if (_bExceptEffect)
        return Result;

    Result += BuffedStat[_type];
    if (Result <= DebuffedStat[_type])
        return 0;

    Result -= DebuffedStat[_type];
    return Result;
}

bool UStatComponent::TryUseResource(
    ECharacterResourceType _type,
    uint32 _amount)
{
    FCharacterResource* pResource = &Resource[_type];

    if (pResource->Value < _amount)
        return false;

    pResource->Value -= _amount;
    pResource->InvokeDelegate();

    return true;
}

void APlayerCharacter::Init()
{
    ...
    // 스탯 초기화
    StatComp->Init(PlayerManager->GetStat(), PlayerManager->GetEquipmentStat());
    StatComp->StartStaminaRecovery();
    StatComp->OnDead.AddUObject(this, &APlayerCharacter::OnDead);
    ...
    // UI 초기화
    if (TObjectPtr<APlayerCombatController> CombatController = Cast<APlayerCombatController>(GetController()))
    {
        ...
        // HPBar UI 이벤트 매핑
        StatComp->GetResourceEvent(ECharacterResourceType::HEALTH).AddUObject(
            StatusBarUI, &UUMPlayerStatusBar::SetHealthBarPercent);
        StatComp->GetResourceEvent(ECharacterResourceType::STAMINA).AddUObject(
            StatusBarUI, &UUMPlayerStatusBar::SetStaminaBarPercent);
        StatComp->GetResourceEvent(ECharacterResourceType::SKILL).AddUObject(
            StatusBarUI, &UUMPlayerStatusBar::SetSkillBarPercent);

        StatComp->OnEffectRegistered.AddUObject(StatusBarUI, &UUMPlayerStatusBar::RegisterStatEffect);
        StatComp->OnEffectRemoved.AddUObject(StatusBarUI, &UUMPlayerStatusBar::RemoveStatEffect);
        ...
    }
    ...
}
```

핵심 기능 1. 플레이어

UEffect

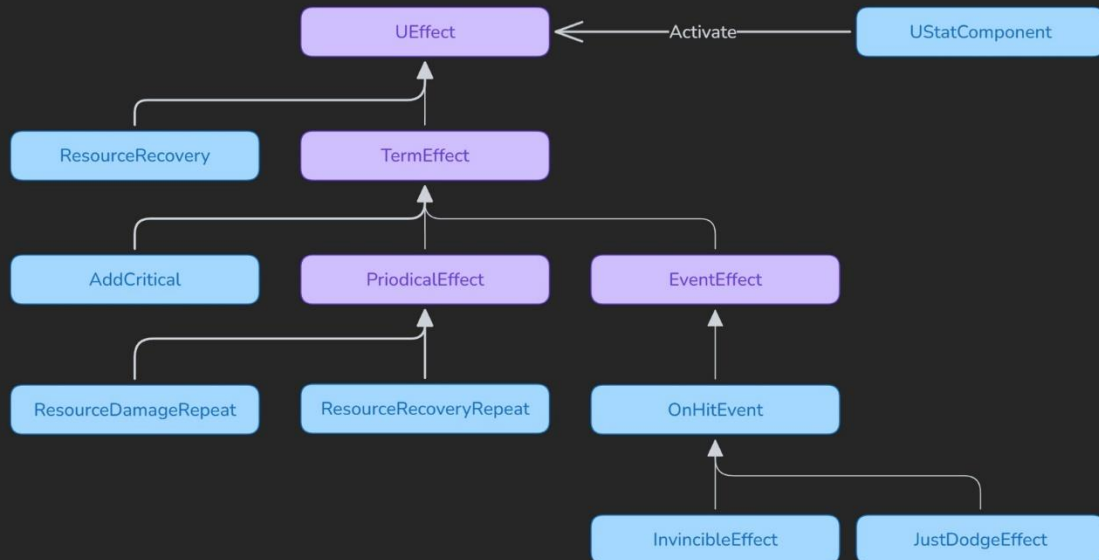
버프, 디버프를 적용하는 효과 클래스입니다.
효과 기능들이 스텟에 영향을 준다는 공통점에 착안하여
UStatComponent에서 모든 효과를 등록, 해제하고 관리하도록 구현했습니다.

UStatComponent를 외부에서 직접 접근하는 과정에서 특정 액터에 대한 의존성이 발생하지 않도록
IEffectHandler 인터페이스를 만들어 ApplyEffect()를 구현하도록 했습니다.

새로운 효과를 추가하기 쉽도록 다형성을 활용했습니다.
상위 클래스의 Activate()를 호출하면
하위 클래스에서 재정의한 구체적인 효과가 실행되도록 구조를 잡았습니다.

주요 하위 클래스

TermEffect : 일정 기간 동안 유지되어 효과를 부여 (예, 10초 동안 적용)
PeriodicalEffect : 일정 기간 동안, 특정 주기마다 효과를 부여 (예, 10초 동안 1초마다 적용)
EventEffect : 액터에게 특정 이벤트가 발생할 때 효과를 부여 (예, 피격 시 발동)



```
struct FApplyEffectParam
{
public:
    TWeakObjectPtr<AActor> Subject;
    TObjectPtr<class UEffectData> EffectData;
    uint32 AddictiveValue{0};
};

class ARPG_HUNTER_API IEffactable
{
    GENERATED_BODY()
public:
    virtual const TObjectPtr<class UStatComponent> GetStatComp() = 0;
    virtual void ApplyEffect(const FApplyEffectParam& _param) = 0;
};
```

```
void UStatComponent::ApplyEffect(const FApplyEffectParam& _param)
{
    if(_param.EffectData->bIsDebuff) // 디버프인 경우 확인
    {
        uint8 HitBit = static_cast<uint8>(EHitOption::NONE);
        uint32 DummyDamage = 0;
        OnHitEvent.Broadcast(HitBit, DummyDamage); // 피격 관련 이벤트드 확인

        // 피격 면역, 디버프 면역인 경우 return;
        uint8 HitMask =
            static_cast<uint8>(EHitOption::IMMUNE_HIT) | static_cast<uint8>(EHitOption::IMMUNE_DEBUF);
        if (CheckHitOptionMask(HitBit, HitMask))
            return;
    }

    TObjectPtr<UEffect> EffectInst = NewObject<UEffect>(this, _param.EffectData->Effect);

    EffectContext Context
    {
        _param.EffectData,
        & _param.EffectData->Param,
        _param.AddictiveValue // 스텟 형성에 필요한 효과 승가량
    };
    EffectInst->Init(_param.Subject, this, &Context);
    EffectInst->Activate();
}
```

```
UCLASS(Abstract)
class ARPG_HUNTER_API UEffect : public UObject
{
    GENERATED_BODY()
    ...
public:
    virtual void Init(
        TWeakObjectPtr<AActor> _subject,
        TWeakObjectPtr<UStatComponent> _target,
        FEffectContext* _context)
    {
        Subject = _subject;
        TargetComp = _target;
        DataPointer = _context->DataPointer;
        BaseParam = _context->Param;
        AddictiveValue = _context->AddictiveValue;
        Stack = 1;
    }

    //하위 클래스에서 구체적 동작 구현
    virtual bool Activate() { return true; };
    virtual void Deactivate() {};
    ...
};
```

```
bool URecoverHealth::Activate()
{
    if (Super::Activate() == false)
        return false;

    if (IsValid())
        GetTarget()->RecoverResource(
            ECharacterResourceType::HEALTH,
            GetValue()
        );

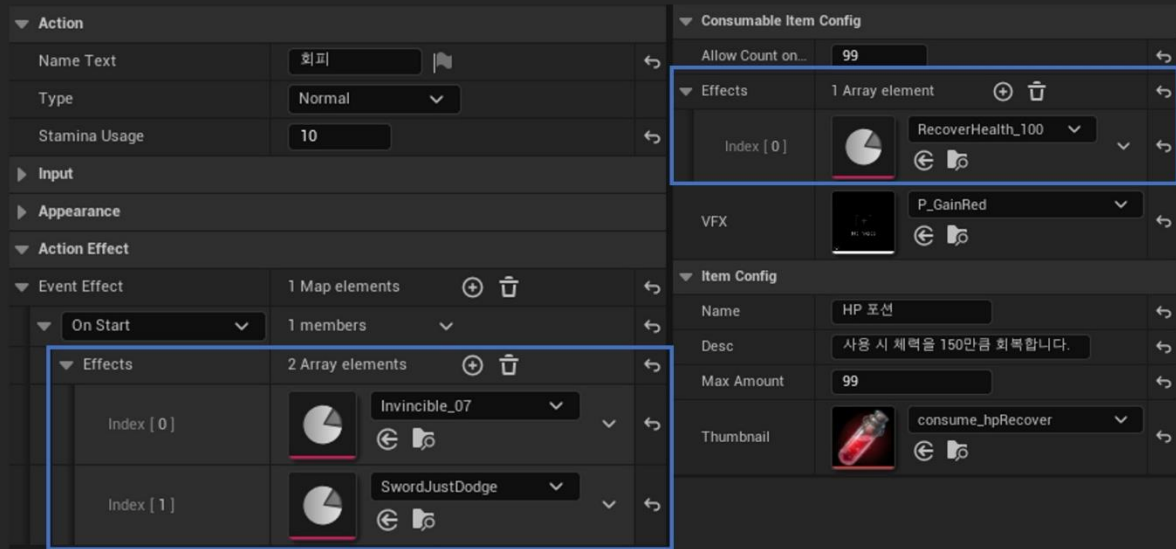
    return true;
}
```

핵심 기능 1. 플레이어

UEffectData

효과 값을 설정해두기 위한 데이터 에셋입니다.
설정해둔 데이터 에셋을 액션 에셋이나 아이템에 할당할 수 있습니다.

동일한 유형의 효과를 데이터 에셋으로 모듈화하고 여러 액션에서 재사용하여
캐릭터 무기의 컨셉에 맞게 조립하듯 사용할 수 있습니다.



```
bool UPlayerActionComponent::PlayAttackAction(EAttackType _type)
{
    // 액션 처리
    ...
    // 액션 시작 시, 효과 발동
    if (AppliedGraph.Actions[CurAttackActionID]->IsContainEventEffect(EActionEvent::ON_START))
        ActivateActionInstanceEffect(
            EActionEvent::ON_START,
            GetOwner(),
            AppliedGraph.Actions[CurAttackActionID]
        );

    return true;
}
```

```
void UPlayerActionComponent::ActivateActionInstanceEffect(
    EActionEvent _type,
    TSharedPtr<AActor> _target,
    TWeakObjectPtr<UActionInstance> _actionInst)
{
    IEffactable* Effectable = Cast<IEffactable>(_target);
    if (Effectable == nullptr)
        return;

    for (const TSharedPtr<class UEffectData>& effectData : _actionInst->GetEventEffect(_type))
    {
        FApplyEffectParam Param;
        Param.Subject = GetOwner();
        Param.EffectData = effectData;
        Param.AddictiveValue = _actionInst->GetAddictiveEffectValue(effectData);

        Effectable->ApplyEffect(Param);
    }
}
```

```
void UConsumableItem::Consume(IEffactable* _target)
{
    TSharedPtr<UConsumableItemConfig> ConsumableConfig = Cast<UConsumableItemConfig>(GetConfig());
    for (const TSharedPtr<UEffectData> Effect : ConsumableConfig->Effects)
    {
        FApplyEffectParam Param;
        Param.EffectData = Effect;

        _target->ApplyEffect(Param);
    }
}
```

핵심 기능 1. 플레이어

플레이어 기능 : 액션과 콤보 연계

UPlayerActionComponent

플레이어의 회피, 상호작용, 공격 등의 애니메이션을 처리하는 컴포넌트입니다.
캐릭터의 여러 동작을 하나의 액션으로 모듈화하여 액션과 이어진 액션을 연계하는 기능을 구현했습니다.

액션 실행에 필요한 스테미너 소모 로직은 UStatComponent에 직접적인 참조를 피하기 위해
델리게이트로 처리했습니다.

APlayerCharacter에서 UStatComponent의 TryUseStamina()를 바인딩 해주었고
액션 실행 전 델리게이트를 호출하여 사용 가능 여부를 확인하는 구조입니다.

콤보 연계

공격 액션은 3가지 유형(일반 공격, 스매시 공격, 스킬)이 있고 각 유형의 입력을 받아 처리합니다.
액션들 간에 사전에 설정된 연결 정보를 기반으로 연속적으로 액션을 수행합니다.

공격 액션이 실행되고 2초간 연결된 공격유형에 해당하는 입력이 들어오면
다음 공격이 실행되는 방식입니다.

▼ Index [3]		1 members
▼ Edge		3 Map elements
	▼ Normal ▼	2 members
	Index	0
	Is Optional	☐
	▼ Smash ▼	2 members
	Index	8
	Is Optional	☐
	▼ Skill ▼	2 members
	Index	10
	Is Optional	☒

```
UENUM(BlueprintType)
enum class EAttackType : uint8
{
    NORMAL    UMETA(DisplayName = "Normal"),
    SMASH     UMETA(DisplayName = "Smash"),
    SKILL     UMETA(DisplayName = "Skill"),

    END       UMETA(Hidden)
};
```

콤보 입력

베기 3

연계

베기 4

회전 베기 1

후퇴 베기

```
void APlayerCharacter::Init()
{
    ...
    // 무기에 따른 애니메이션 및 액션 초기화
    TObjectPtr<UAnimInstance> AnimInst = GetMesh()->GetAnimInstance();

    ActionComp->Init({
        PlayerManager->GetWeaponConfig(),
        AnimInst,
        MapEquipmentMeshComp[EEquipmentType::WEAPON],
        PlayerManager->GetSkillDevelop()->GetSkillSelectPtr()
    });
    ActionComp->StaminaUsagePredicate.BindUObject(StatComp, &UStatComponent::TryUseStamina);
    ...
    // UI 초기화
    if (TObjectPtr<APlayerCombatController> CombatController = Cast<APlayerCombatController>(GetController()))
    {
        ...
        TObjectPtr<UWActionGuide> ActionGuideUI = CombatHUD->GetPlayerUI()->GetActionGuide();
        ActionComp->OnActionUpdated.BindUObject(ActionGuideUI, &UWActionGuide::SetActionInfo);
        ActionComp->ResetAction();
    }
    ...
}
```

```
bool UPlayerActionComponent::PlayAttackAction(EAttackType _type)
{
    if (IsValidAttackInput(_type) == false)
        return false;

    uint8 id = !IsInAttackCombo() ?
        AppliedGraph.GraphStart[_type].Index :
        AppliedGraph.Graph[CurAttackActionID][_type].Index;

    TweakObjectPtr<UAction> ActionData = AppliedGraph.Actions[id]->GetAction();

    if (StaminaUsagePredicate.IsBound() &&
        StaminaUsagePredicate.Execute(AppliedGraph.Actions[id]->GetStaminaUsage()) == false)
        return false;

    CurAttackActionID = id;
    CurActionProcess = EActionProcess::START;
    CurActionInput = ActionData->InputType;
    BroadcastActionUpdated();

    GetAnimInstance()->Montage_Play(ActionData->Montage);

    ClearActionResetTimer(); // 이전 콤보에 대한 리셋 타이머 클리어

    // 액션 시작 시, 효과 발동
    if (AppliedGraph.Actions[id]->IsContainEventEffect(EActionEvent::ON_START))
        ActivateActionInstanceEffect(EActionEvent::ON_START, GetOwner(), AppliedGraph.Actions[id]);

    return true;
}
```

핵심 기능 1. 플레이어

UAction

액션을 모듈화한 데이터 에셋입니다.

액션으로 실행할 UAnimMontage를 기반으로 작동합니다.

UAnimMontage에서 설정한 노티파이를 이용하여 액션의 세부적인 동작을 수행합니다.

공격 액션의 경우, UAnimNotify를 상속한 UAttackNotify를 사용합니다.

UAttackNotify의 멤버변수 인덱스를 통해 원하는 시점에 수행하는 동작에 대한 데이터에 접근합니다.

UAttackNotify의 경우, 플레이어 뿐만 아니라 몬스터도 사용하고 있기 때문에

특정 액터 클래스에 의존하지 않도록 IAttackNotifyHandler 인터페이스를 만들어 사용했습니다.

HandleAttackNotify()를 구현하여 몽타주 재생 중 정확한 시점의 타격 및 세분화된 공격 동작을 수행합니다.

The image illustrates the implementation of a player attack action, showing the integration of animation, notification, and combat logic.

UAction Properties:

- Name Text: 진 회전 베기
- Type: Smash
- Stamina Usage: 15
- Input Type: Normal
- Montage: AM_Sword_Attack_4.2
- Arr Option: 2 Array elements
- Effect: SubObject
- Action Effect: 1 Map elements
- Event Effect: On Start (1 members)
- Effects: 1 Array element (UsingSkill_CriticalEffect)

UAttackNotify Properties:

- Notify: (empty)
- Attack Notify: Option 1
- Notify Color: (pink)
- Advanced: (empty)

C++ Code Snippets:

```
void UAttackNotify::Notify(USkeletalMeshComponent* MeshComp, UAnimSequenceBase* Animation, const FAnimNotifyEventReference& EventReference)
{
    Super::Notify(MeshComp, Animation, EventReference);

    IAttackNotifyHandler* Handler = Cast<IAttackNotifyHandler>(MeshComp->GetOwner());
    if (Handler)
        Handler->HandleAttackNotify(Option);
}
```

```
class ARPG_HUNTER_API IAttackNotifyHandler
{
    GENERATED_BODY()
public:
    virtual void HandleAttackNotify(uint8 _opt) = 0;
};
```

```
void APlayerCharacter::HandleAttackNotify(uint8 _opt)
{
    TWeakObjectPtr<APlayerCharacter> WeakThis(this);

    uint8 OptionIdx = _opt;
    ActionComp->ProcessAttack(_opt, ECC_GameTraceChannel4, [WeakThis, OptionIdx](TArray<FHitResult>& _hitResults)
    {
        // 피격 처리
        ...
    });
}
```

Animation Timeline (AM_Sword_Attack_4.2):

- Timeline: 0 to 110 seconds.
- Montage (DefaultGroup): CS_Combo_Attack_02_All_Seq, CS_Combo_Attack_02_02_Seq.
- Notifies: 1 (AttackNotify).
- Events: 1 (AttackNotify), 2 (AttackNotify), 3 (AttackNotify), 4 (AttackNotify), 5 (AttackNotify), 6 (AttackNotify), 7 (AttackNotify), 8 (AttackNotify), 9 (AttackNotify), 10 (AttackNotify), 11 (ActionProcessNotify).
- Effects: NS_Trail_5c, Sword_Slice_Cue.

핵심 기능 1. 플레이어

액션 실행 중 효과 발동

액션이 진행되는 과정에서 효과를 적용할 수 있도록 했습니다.

UAction에 원하는 시점에 UEffectData를 할당하여 효과를 적용할 수 있습니다.

액션에서 효과를 적용하는 시점을 4가지로 설정해두고 각각의 시점에서 할당한 효과를 적용시킵니다.

```
UENUM(BlueprintType)
enum class EActionEvent : uint8
{
    ON_START
    ON_PROGRESS
    ON_HIT
    ON_ENEMY_HIT
};
```

▼ Action Effect	
▼ Event Effect	2 Map elements
▼ On Start	1 members
▼ Effects	1 Array element
Index [0]	UsingSkill_Invincible
▼ On Hit	1 members
▼ Effects	1 Array element
Index [0]	RecoverStamina_10

UActionComboData

캐릭터가 사용하는 UAction간의 연계에 대한 정보를 담은 데이터 에셋입니다.

UAction간의 연결은 인접리스트를 이용한 그래프로 구현했습니다.

▼ Action Combo Data	
▼ Attack Action Array	14 Array elements
Index [0]	SwordNormalAttack1
Index [1]	SwordNormalAttack2
Index [2]	SwordNormalAttack3
Index [3]	SwordNormalAttack4
Index [4]	SwordSmashAttack1_1

▼ Graph	
▼ Index [0]	1 members
▼ Edge	3 Map elements
▼ Normal	2 members
Index	1
Is Optional	
▼ Smash	2 members
Index	4
Is Optional	
▼ Skill	2 members
Index	10
Is Optional	☒

▼ Graph	
▼ Index [1]	1 members
▼ Edge	3 Map elements
▼ Normal	2 members
Index	2
Is Optional	
▼ Smash	2 members
Index	5
Is Optional	
▼ Skill	2 members
Index	10
Is Optional	☒

```
void UPlayerActionComponent::ProcessAttack(
    uint8 _opt,
    ECollisionChannel _traceChannel,
    TFunction<void(TArray<FHitResult>&)> _onHitAction, TWeakObjectPtr<AActor> _target)
{
    Super::ProcessAttack(_opt, _traceChannel, _onHitAction, _target);

    // 공격 처리
    ...
    // 공격 히트 시, 효과 발동
    if (_onHitAction)
        _onHitAction(HitResults);

    // 자기 버프 적용
    if (CurAction->IsContainEventEffect(EActionEvent::ON_HIT))
        ActivateActionInstanceEffect(EActionEvent::ON_HIT, GetOwner(), CurAction);

    // 적에게 디버프 적용
    for (const FHitResult& Result : HitResults)
    {
        if (CurAction->IsContainEventEffect(EActionEvent::ON_ENEMY_HIT))
            ActivateActionInstanceEffect(EActionEvent::ON_ENEMY_HIT, Result.GetActor(), CurAction);
        ...
    }
}
```

```
bool UPlayerActionComponent::PlayAttackAction(EAttackType _type)
{
    if (IsValidAttackInput(_type) == false)
        return false;

    uint8 id = !IsInAttackCombo() ?
        AppliedGraph.GraphStart[_type].Index :
        AppliedGraph.Graph[CurAttackActionID][_type].Index;

    TWeakObjectPtr<UAction> ActionData = AppliedGraph.Actions[id]->GetAction();

    if (StaminaUsagePredicate.IsBound() &&
        StaminaUsagePredicate.Execute(AppliedGraph.Actions[id]->GetStaminaUsage()) == false)
        return false;

    CurAttackActionID = id;
    CurActionProcess = EActionProcess::START;
    CurActionInput = ActionData->InputType;
    BroadcastActionUpdated();

    GetAnimInstance()->Montage_Play(ActionData->Montage);

    ClearActionResetTimer(); // 이전 콤보에 대한 리셋 타이머 클리어

    // 액션 시작 시, 효과 발동
    if (AppliedGraph.Actions[id]->IsContainEventEffect(EActionEvent::ON_START))
        ActivateActionInstanceEffect(EActionEvent::ON_START, GetOwner(), AppliedGraph.Actions[id]);

    return true;
}
```

핵심 기능 1. 플레이어

플레이어 기능 : 스킬

플레이어 스킬 구현

플레이어의 스킬은 본래 가진 액션의 성능을 강화하거나 새로운 액션 연계를 얻는 방향으로 구현했습니다. 스킬 성장에 필요한 스킬 포인트는 스테이지를 클리어해서 얻은 경험치로 레벨을 올려 얻을 수 있습니다. 스킬 포인트로 스킬트리에 따라 원하는 대로 육성할 수 있도록 개발했습니다.

UActionInstance

스킬의 적용은 플레이어의 선택에 따라 동적이기 때문에 UPlayerActionComponent에서 UAction을 한 차례 감싼 UActionInstance 클래스를 사용하고 있습니다. UAction의 기본 설정값과 함께 스킬 성장으로 얻은 값을 추가하도록 처리했습니다.

USkillUpgrade

스킬 강화를 나타내는 데이터 에셋입니다. 플레이어가 해당 강화를 선택하면 지정된 액션에 강화 내용이 반영됩니다. 다형성을 이용하여 상위 클래스에서 AdjustSkillNode()를 호출하면 하위 클래스에서 재정의한 내용을 스킬에 반영하는 식으로 구조화했습니다.

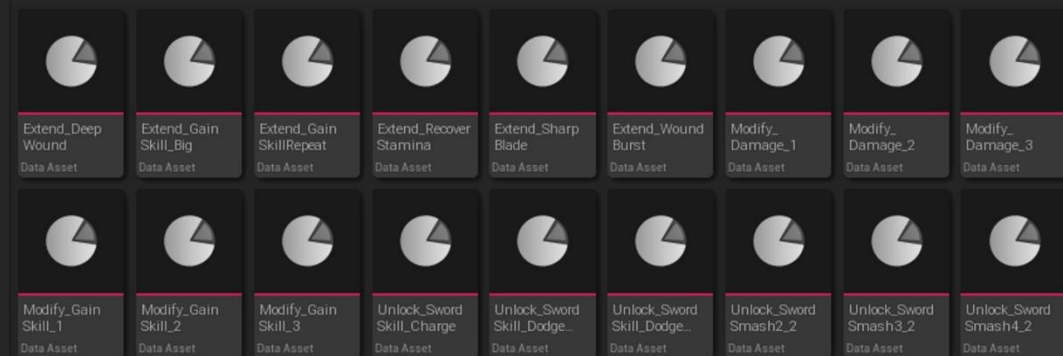
주요 하위 클래스

USkillNodeUnlockAction : 특정 액션의 연계를 해금

USkillNodeModifyEffect : 액션이 기존에 가진 이펙트(버프, 디버프 등)의 수치를 변경

USkillNodeExtendEffect : 액션에 새로운 이펙트를 추가하는 기능

USkillNodeModifySpec : 액션의 데미지량, 스테미너 소모량 등을 변경



```
void UPlayerActionComponent::Init(const FPlayerActionInitParam& _param)
{
    // 콤보 그래프 구성
    ...

    // 플레이어가 설정한 스킬 정보 반영
    FAdjustParam SkillAdjustParam;
    SkillAdjustParam.ActionArray = &AppliedGraph.Actions;
    SkillAdjustParam.Graph = &AppliedGraph.Graph;
    SkillAdjustParam.GraphStart = &AppliedGraph.GraphStart;

    TSharedPtr<USkillTreeData> SkillTree = CurWeapon->SkillTree;
    if (nullptr == SkillTree)
    {
        ResetAction();
        return;
    }

    for (const TPair<uint8, TMap<uint8, int8>>& TreePair : (*_param.SkillDevelop))
    {
        for (const TPair<uint8, int8>& NodePair : TreePair.Value)
        {
            if (NodePair.Value < 0)
                continue;

            const FSkillNode* SkillNode = SkillTree->SkillTrees[TreePair.Key].GetNode(NodePair.Key);
            if (SkillNode->UpgradeInfos.Num() <= NodePair.Value)
                continue;

            const FUpgradeInfo& UpgradeInfo = SkillNode->UpgradeInfos[NodePair.Value];
            UpgradeInfo.Upgrade->AdjustSkillNode(UpgradeInfo.TargetIndex, SkillAdjustParam);
        }
    }

    ResetAction();
}
```

```
void USkillNodeModifyEffect::AdjustSkillNode(uint8 _targetIdx, FAdjustParam& _param)
{
    Super::AdjustSkillNode(_targetIdx, _param);

    (*_param.ActionArray)[_targetIdx]->AddEffectValue(TargetEffect->GetClass(), AddictiveValue);
}
```

```
void USkillNodeExtendEffect::AdjustSkillNode(uint8 _targetIdx, FAdjustParam& _param)
{
    Super::AdjustSkillNode(_targetIdx, _param);

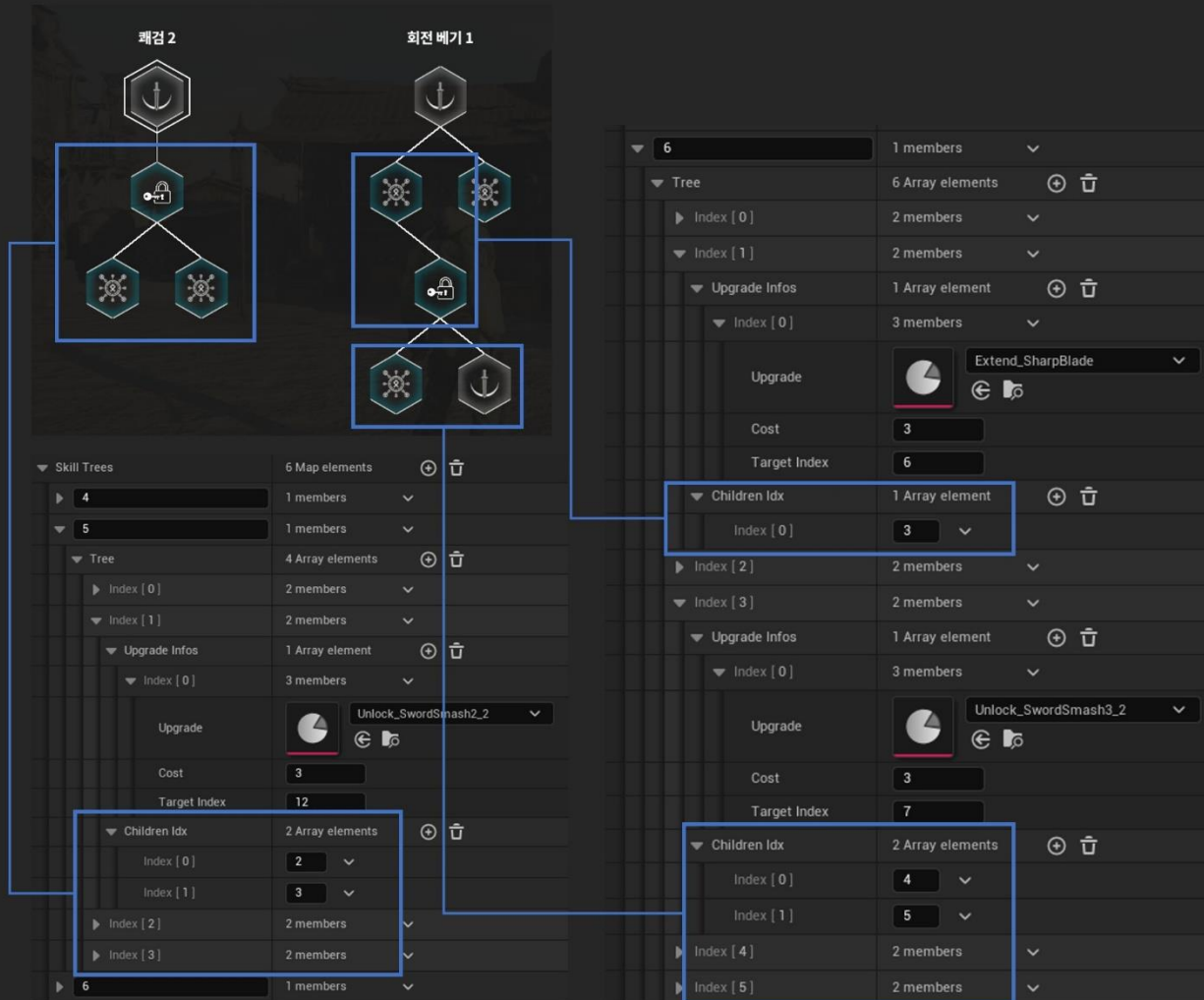
    TSharedPtr<UActionInstance> ActionInst = (*_param.ActionArray)[_targetIdx];

    for (const TPair<EActionEvent, FActionEventEffect>& Pair : ExtendEffects)
    {
        for (TSharedPtr<UEffectData> EffectData : Pair.Value.Effects)
            ActionInst->ExtendEventEffect(Pair.Key, EffectData);
    }
}
```

핵심 기능 1. 플레이어

USkillTreeData

플레이어가 선택할 수 있는 스킬들을 트리 형태로 설정한 데이터 에셋입니다. 강화할 액션의 내용, 스킬 포인트 소모량, 강화 내용으로 노드를 구성하고 연결될 자식 노드들의 인덱스를 설정하여 트리 형태가 되도록 설정했습니다.



```
class USkillUpgrade;

USTRUCT()
struct FUpgradeInfo
{
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere)
    TObjectPtr<USkillUpgrade> Upgrade;
    UPROPERTY(EditAnywhere)
    uint8 Cost;
    /// 업그레이드를 반영할 액션의 인덱스
    UPROPERTY(EditAnywhere)
    uint8 TargetIndex;
};

USTRUCT()
struct FSkillNode
{
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere)
    TArray<FUpgradeInfo> UpgradeInfos; // 이 노드가 나타내는 업그레이드 기능

    UPROPERTY(EditAnywhere, Category = "Children")
    TArray<uint8> ChildrenIdx; // SkillTree의 배열에 할당된 노드들의 인덱스 사용
};

USTRUCT()
struct FSkillTree
{
    GENERATED_BODY()

public:
    /// 노드들은 전체적으로 하나의 배열로 관리
    /// 각 노드에 자식에 해당하는 노드들의 포인터 대신 인덱스를 기입할 것
    UPROPERTY(EditAnywhere)
    TArray<FSkillNode> Tree;
};

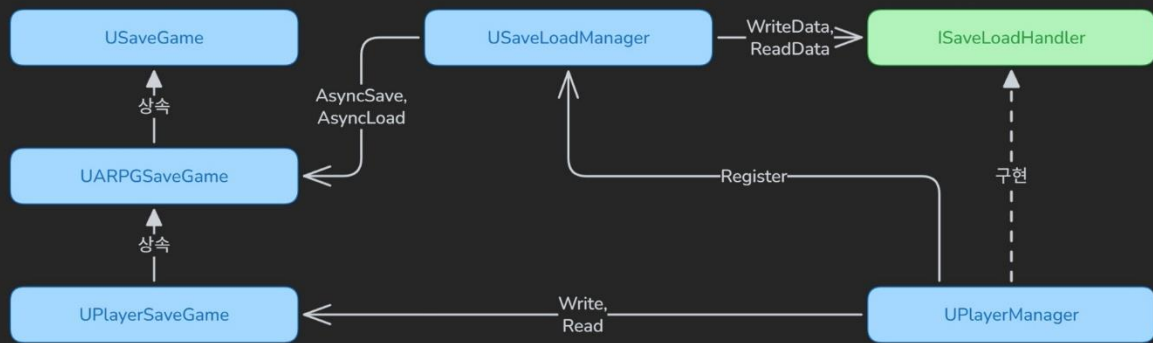
UCLASS()
class ARPG_HUNTER_API USkillTreeData : public UDataAsset
{
    GENERATED_BODY()
public:
    /// Key : ActionComboData 중, AttackActionArray의 인덱스
    UPROPERTY(EditAnywhere)
    TMap<uint8, FSkillTree> SkillTrees;
};
```

핵심 기능 1. 플레이어

플레이어 데이터 저장

USaveGame 상속한 UPlayerSaveGame 구현

플레이어의 레벨, 경험치, 보유한 재화, 스킬 정보등을 저장합니다.
실질적인 저장 동작은 USaveLoadManager를 만들었고 ISaveLoadHandler 인터페이스를 이용해서
플레이어 데이터 외에도 다른 정보들을 저장하기 쉽도록 구조화했습니다.



```
class UARPGSaveGame;

class ARPG_HUNTER_API ISaveLoadHandler
{
    GENERATED_BODY()

public:
    virtual void WriteSaveData(UARPGSaveGame* _savegame) = 0;
    virtual void ReadSaveData(UARPGSaveGame* _savegame) = 0;
};
```

```
void UPlayerManager::WriteSaveData(UARPGSaveGame* _savegame)
{
    UPlayerSaveGame* PlayerSave
        = Cast<UPlayerSaveGame>(_savegame);

    // 데이터 쓰기
    ...
}
```

```
void UPlayerManager::ReadSaveData(UARPGSaveGame* _savegame)
{
    TObjectPtr<UPlayerSaveGame> PlayerSaveData
        = Cast<UPlayerSaveGame>(_savegame);

    // 데이터 읽기
    ...
}
```

```
template<typename ARPGSaveGameType>
inline void USaveLoadManager::SaveGame(ISaveLoadHandler* _handler, FOnSaveLoadComplete& _callback)
{
    UClass* Key = ARPGSaveGameType::StaticClass();
    if (nullptr == MapHandler.Find(Key))
        return;

    UARPGSaveGame* SaveGame = CreateGetSaveGameObject(Key);
    MapHandler[Key]->WriteSaveData(SaveGame);

    AsyncSaveGame(SaveGame, _callback);
}

template<typename ARPGSaveGameType>
inline void USaveLoadManager::LoadGame(FOnSaveLoadComplete& _callback)
{
    UClass* Key = ARPGSaveGameType::StaticClass();
    if (nullptr == MapHandler.Find(Key))
        return;

    AsyncLoadGame(Key, ARPGSaveGameType::SlotName, DEFAULT_SLOT_INDEX, _callback);
}
```

```
void USaveLoadManager::AsyncSaveGame(UARPGSaveGame* _savegame, FOnSaveLoadComplete& _callback)
{
    FAsyncSaveGameToSlotDelegate Callback;
    Callback.BindWeakLambda(this,
        [_callback](const FString& _slotName, int32 _userIndex, bool _bIsIgnore)
        {
            _callback.ExecuteIfBound();
        }
    );

    UGameplayStatics::AsyncSaveGameToSlot(_savegame, _savegame->SlotName, DEFAULT_SLOT_INDEX, Callback);
}
```

```
void USaveLoadManager::AsyncLoadGame(
    UClass* _key,
    const FString& _slotName,
    int32 _slotIndex,
    FOnSaveLoadComplete& _callback)
{
    FAsyncLoadGameFromSlotDelegate Callback;
    Callback.BindWeakLambda(this,
        [this, _key, _callback](const FString& _slotName, int32 _userIndex, USaveGame* _data)
        {
            TObjectPtr<UARPGSaveGame> SaveGame = Cast<UARPGSaveGame>(_data);
            MapHandler[_key]->ReadSaveData(SaveGame);

            _callback.ExecuteIfBound();
        }
    );

    UGameplayStatics::AsyncLoadGameFromSlot(_slotName, DEFAULT_SLOT_INDEX, Callback);
}
```

핵심 기능 2. 몬스터

몬스터 기능

몬스터 공통사항과 몬스터 데이터

상속 및 구현
플레이어와 같이 ACharacter, IEffactable, IHitable, IAttackNotifyHandler를 상속하고 구현하고 있습니다.

UStatComponent

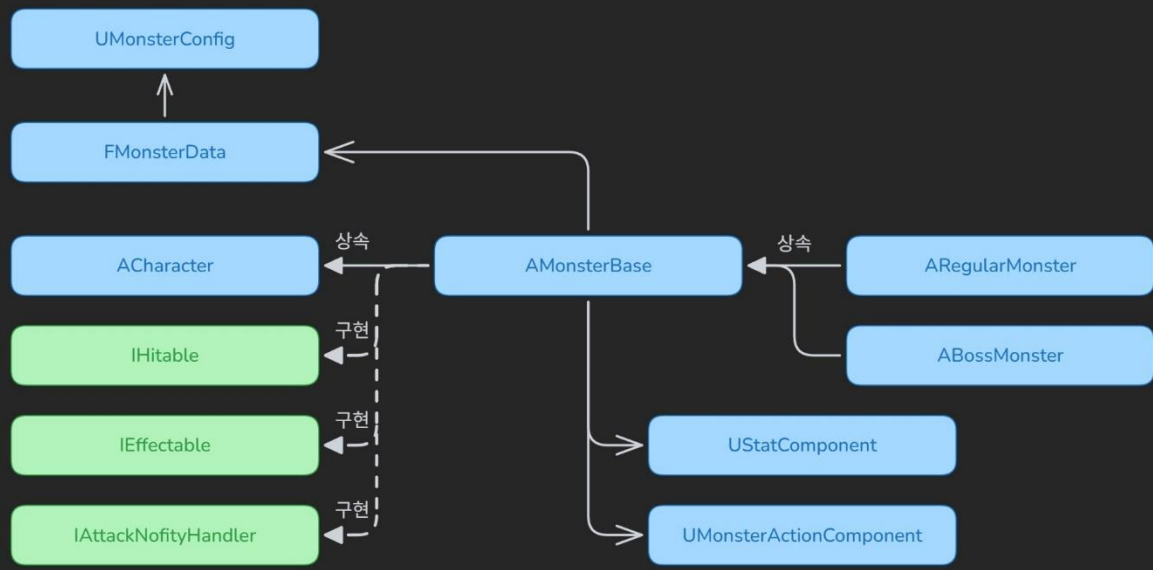
플레이어와 동일하게 스탯 기능과 이펙트를 반영할 수 있도록 했습니다.

MonsterActionComponent

몬스터의 경우, 플레이어와 다르게 콤보 연계의 기능이 필요 없으므로 간략한 형태로 구현했습니다.
BT를 통해서 입력된 공격 액션을 수행합니다.

MonsterConfig, MonsterData

몬스터의 경우, 외형, 액션은 데이터에셋으로, 스탯은 데이터테이블을 통해서 구성하도록 구조화했습니다.
몬스터 데이터테이블의 ID를 가지고 던전 스테이지에 사용할 몬스터들을 구성할 수 있도록 했습니다.



```
UCLASS(Abstract)
class ARPG_HUNTER_API AMonsterBase :
public ACharacter,
public IHitable,
public IEffactable,
public IAttackNotifyHandler
{
    GENERATED_BODY()

public:
    AMonsterBase();
    ...

private:
    UPROPERTY(EditAnywhere)
    TObjectPtr<class UStatComponent> StatComp;
    ...

protected:
    UPROPERTY(EditAnywhere)
    TObjectPtr<class UMonsterActionComponent> ActionComp;
    ...

public:
    // IAttackNotifyHandler(를) 통해 상속됨
    virtual void HandleAttackNotify(uint8 _opt) override;
    // IHitable(를) 통해 상속됨
    virtual void HitBy(const FHitInfo& _hitInfo) override;

    // IEffactable(를) 통해 상속됨
    const TObjectPtr<class UStatComponent> GetStatComp() override { return StatComp; }
    virtual void ApplyEffect(const FApplyEffectParam& _param) override;
}
```

Row Name	Base Stat
1001	("Health" = 100, "Stamina" = 100, "Skill" = 0, "Attack" = 75, "Defense" = 50, "Critical Per" = 0, "Critical Damage Per" = 0)
1002	("Health" = 100, "Stamina" = 100, "Skill" = 0, "Attack" = 75, "Defense" = 50, "Critical Per" = 0, "Critical Damage Per" = 0)
1003	("Health" = 5000, "Stamina" = 1000, "Skill" = 10, "Attack" = 75, "Defense" = 50, "Critical Per" = 0, "Critical Damage Per" = 0)

핵심 기능 2. 몬스터

일반 몬스터

BehaviorTree

BehaviorTree를 활용하여 행동 양식을 구현했습니다.

일반 상태

주변을 돌아다니며 탐색합니다.

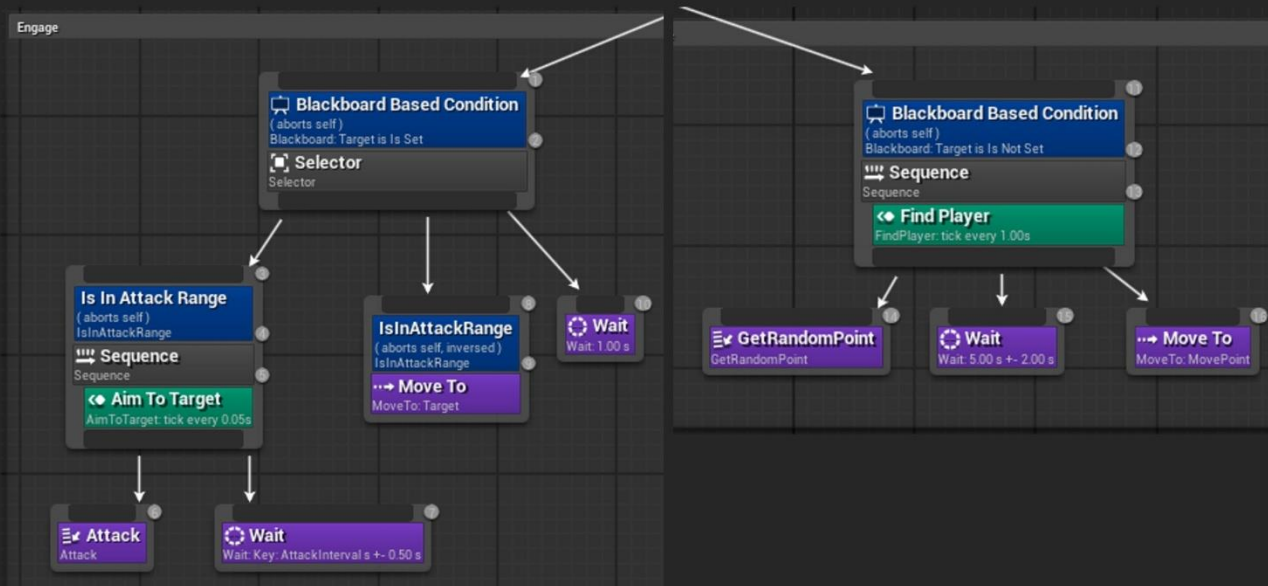
UBTService를 상속한 UFindPlayerService에서 1초 마다 SphereTrace로 주변을 탐색하고 적을 발견하면 공격 상태로 전환합니다.

공격 상태

타겟에게 다가가며 공격 범위 내일 때 공격합니다.

UBTTask를 상속한 UAttackTask에서 몬스터가 가진 액션 중 랜덤으로 공격 액션을 수행합니다.

추적 불가능한 경우 일정 시간 동안 적에게 다가가지 못하는 상황일 때, 다시 일반 상태로 복귀합니다.



```
void UBTService_FindPlayer::TickNode(UBehaviorTreeComponent& OwnerComp, uint8* NodeMemory, float DeltaSeconds)
{
    Super::TickNode(OwnerComp, NodeMemory, DeltaSeconds);

    ...
    UBlackboardComponent* BBComp = OwnerComp.GetBlackboardComponent();
    float RecogRange = BBComp->GetValueAsFloat(FName(TEXT("RecognitionRange")));
    // 주변 조사
    FHitResult HitResult;
    bool IsHit = UKismetSystemLibrary::SphereTraceSingle(
        Monster, Loc, Loc,
        RecogRange,
        UEngineTypes::ConvertToTraceType(ECC_GameTraceChannel3),
        false, {Monster},
        EDrawDebugTrace::None, HitResult, true
    );

    if (IsHit)
    {
        // 타겟 설정
        BBComp->SetValueAsObject(FName(TEXT("Target")), HitResult.GetActor());
        Monster->OnTargetFound();
    }
}
```

```
EBTNodeResult::Type UBTTask_Attack::ExecuteTask(UBehaviorTreeComponent& OwnerComp, uint8* NodeMemory)
{
    Super::ExecuteTask(OwnerComp, NodeMemory);

    AMonsterBase* Owner = Cast(OwnerComp.GetAIOwner()->GetPawn());
    if(Owner == nullptr)
        return EBTNodeResult::Failed;

    UBlackboardComponent* BBComp = OwnerComp.GetBlackboardComponent();
    APlayerCharacter* Target = Cast<APlayerCharacter>(BBComp->GetValueAsObject(FName(TEXT("Target"))));
    if (Target == nullptr)
        return EBTNodeResult::Failed;

    Owner->OnAttackMontageEnded.BindLambda(
        [this, &OwnerComp]()
        {
            UBlackboardComponent* BBComp = OwnerComp.GetBlackboardComponent();
            APlayerCharacter* Target = Cast<APlayerCharacter>(BBComp->GetValueAsObject(FName(TEXT("Target"))));

            if (Target == nullptr || Target->IsDead())
                BBComp->ClearValue(FName(TEXT("Target")));

            FinishLatentTask(OwnerComp, EBTNodeResult::Succeeded);
        }
    );

    float Interval = Owner->Attack(AttackType);
    if (Interval < 0.0f)
        return EBTNodeResult::Failed; // 공격 동작이 유효하지 않은 상황 실패처리

    BBComp->SetValueAsFloat(FName(TEXT("AttackInterval")), Interval); // 공격 후 대기시간

    return EBTNodeResult::InProgress;
}
```

보스 몬스터

BehaviorTree

일반 몬스터와 같이 BehaviorTree를 활용했으며 공격 행동에 관련하여 추가적인 작업을 수행합니다.

일반 상태

주변 랜덤한 곳을 돌아다니며 탐색합니다.

일반 몬스터와 동일하게 FindPlayer를 이용하여 1초마다 주변을 탐색하고 플레이어 발견하면 공격 상태로 전환합니다.

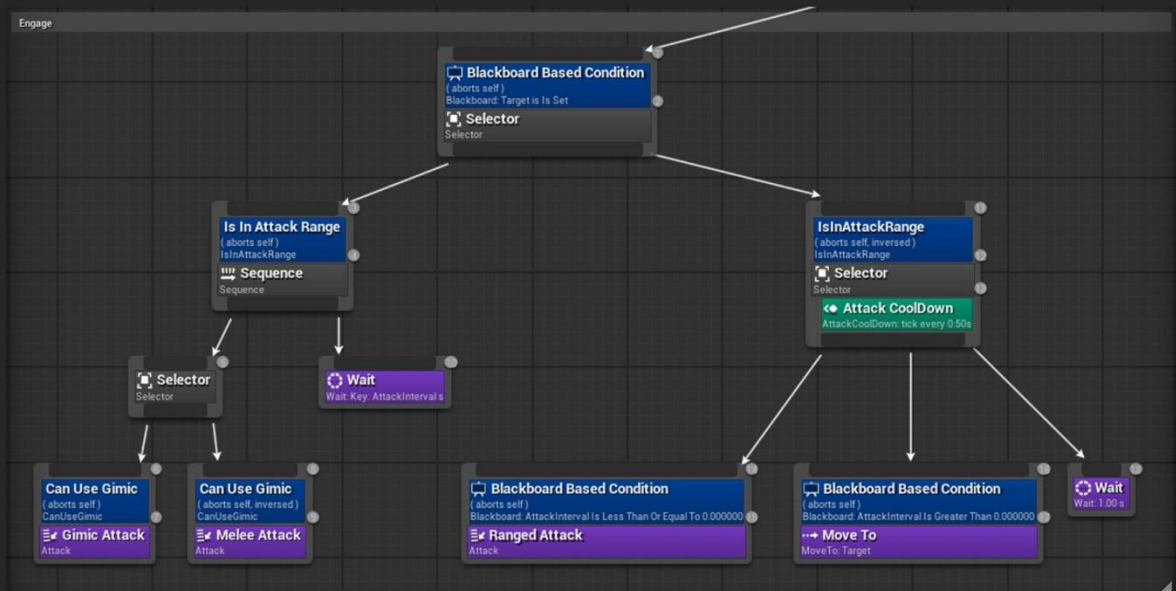
공격 상태

플레이어와의 거리에 따라 근접 공격이 가능한지 확인합니다.

근접 공격이 불가한 경우 타겟에게 다가가며 원거리 공격으로 견제합니다.

근접 공격이 가능하면 일반 공격 중 랜덤으로 한 개를 수행합니다.

스킬 포인트가 10 이상 모이면 기믹을 수행합니다.



```
bool UBTDecorator_IsInAttackRange::CalculateRawConditionValue(
    UBehaviorTreeComponent& OwnerComp,
    uint8* NodeMemory) const
{
    Super::CalculateRawConditionValue(OwnerComp, NodeMemory);

    AMonsterBase* Owner = Cast<AMonsterBase>(OwnerComp.GetAIOwner() -> GetPawn());
    if (Owner == nullptr)
        return false;

    UBlackboardComponent* BBComp = OwnerComp.GetBlackboardComponent();
    APlayerCharacter* Target = Cast<APlayerCharacter>(BBComp -> GetValueAsObject(FName(TEXT("Target"))));
    if (Target == nullptr || Target -> IsDead())
        return false;

    float Range = BBComp -> GetValueAsFloat(FName(TEXT("AttackRange")));
    float DistSqr = FVector::DistSquared(Owner -> GetActorLocation(), Target -> GetActorLocation());

    return DistSqr <= FMath::Square(Range);
}
```

```
bool UBTDecorator_CanUseGimic::CalculateRawConditionValue(
    UBehaviorTreeComponent& OwnerComp,
    uint8* NodeMemory) const
{
    IGimicHandler* GimicHandler = Cast<IGimicHandler>(OwnerComp.GetAIOwner() -> GetPawn());
    if (GimicHandler == nullptr)
        return false;

    return GimicHandler -> CanUseGimic();
}
```

핵심 기능 2. 몬스터

기믹 수행

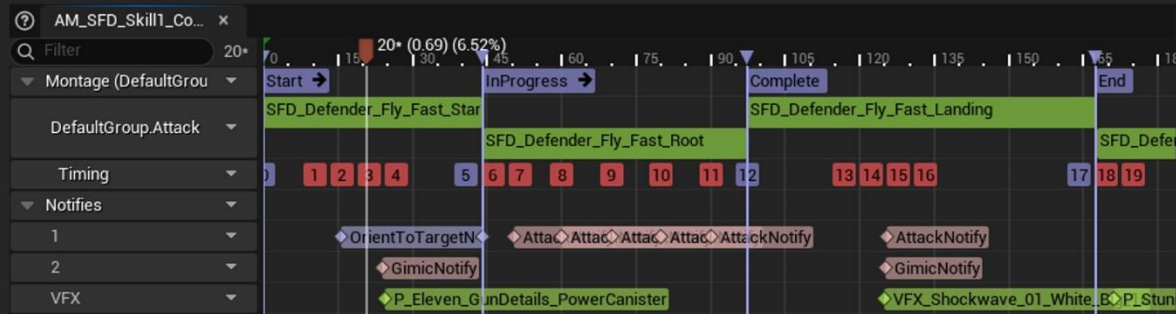
보스 몬스터는 기믹을 수행할 수 있도록 구현했습니다.

공격 몽타주에 GimicNotify를 설정하여 특정한 종류의 기믹을 수행할 수 있습니다.

각 기믹 액션을 파훼하면 보스 몬스터는 일시적으로 그로기 상태가 됩니다.

기믹 액션 종류

- 카운터 : 보스가 정면으로 이동하며 공격합니다.
보스 정면 (보스 기준 120도 이내 방향)에서 스매시, 스킬 공격 시 파훼 가능합니다.
- 무력화 : 보스가 일정 시간 동안 무력화 가능 상태가 됩니다.
무력화 게이지를 0으로 만들지 못하면 일정 시간 후 매우 강력한 공격을 실행합니다.



Gimic Notify		Gimic Notify	
Type	Counter	Type	Stagger
Value	1	Value	200
Notify Color		Notify Color	
Advanced		Advanced	



무력화 기믹 시,
무력화 게이지 활성화

```
void UGimicNotify::Notify(
    USkeletalMeshComponent* MeshComp,
    UAnimSequenceBase* Animation,
    const FAnimNotifyEventReference& EventReference)
{
    Super::Notify(MeshComp, Animation, EventReference);

    if (IGimicHandler* Handler = Cast<IGimicHandler>(MeshComp->GetOwner()))
        Handler->HandleGimicNotify(Type, Value);
}
```

```
void ABossMonster::HandleGimicNotify(EGimicType _type, uint16 _gimicValue)
{
    TObjectPtr<UBossActionComponent> BossAction = Cast<UBossActionComponent>(ActionComp);
    if (_type < EGimicType::END)
    {
        if (GetState() != EMonsterState::NORMAL)
            return;

        UStatComponent* Stat = GetStatComp();
        Stat->TryUseResource(ECharacterResourceType::SKILL,
            Stat->GetResourceMaxValue(ECharacterResourceType::SKILL));

        BossAction->StartGimic(_type, _gimicValue);
        SetState(EMonsterState::GIMIC);
    }
    else
    {
        BossAction->EndGimic();
        SetState(EMonsterState::NORMAL);
    }
}
```

```
void UBossActionComponent::InterruptGimic(const FHitInfo& _hitInfo)
{
    TWeakObjectPtr<UAnimInstance> AnimInst = GetAnimInstance();
    const FMonsterAction& MonsterAction = GetData()->Config->AttackActions[GetCurAttackIdx()];

    if (nullptr == AnimInst || nullptr == MonsterAction.Action->Montage)
        return;

    bool bInterrupted = false;
    switch (CurGimicType)
    {
        case EGimicType::COUNTER:
            bInterrupted = InterruptCounter(_hitInfo);
            break;
        case EGimicType::STAGGER:
            bInterrupted = InterruptStagger(_hitInfo);
            break;
    }

    if (bInterrupted)
    {
        GetAnimInstance()->Montage_JumpToSection(EnumToName(EGimicType::END), MonsterAction.Action->Montage);
        EndGimic();
    }
}
```

핵심 기능 3. 스테이지

스테이지

레벨과 스테이지 데이터

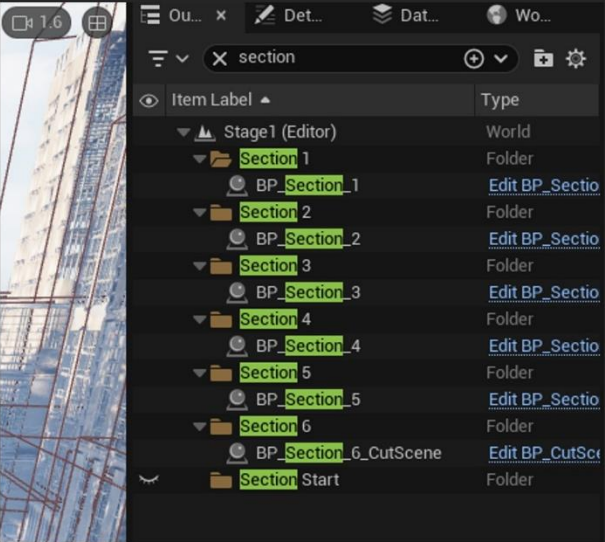
StageData
스테이지에 사용할 몬스터들의 정보와 보상 정보를 데이터테이블로 만들어두었습니다.

StageSection
스테이지는 몇 개의 구역으로 구성되어 있고 모든 구역을 클리어하면 스테이지 종료됩니다.
StageSection은 레벨 배치되어 있고 인덱스를 부여하여 각각의 구역을 표시합니다.

플레이어가 StageSection에 들어서면 설정된 Index를 통해 몬스터들을 생성합니다.
생성된 몬스터들을 모두 쓰러뜨리면 해당 구역이 완료됩니다.

레벨에서 생성할 몬스터들은 각 구역에서 새로 스폰하지 않고
게임시작 시, UObjectPoolManager에 등록하여 재사용합니다.

1001	
▼ 1001	
Stage Name	도시 폐허 1
Stage Desc	폐허가 된 도심을 탐사하세요.
Level to Load	Stage1
Start Point	StartPoint_1
▼ Sections	3 Array elements
▶ Index [0]	1 members
▶ Index [1]	1 members
▼ Index [2]	1 members
▼ Spawn	2 Array elements
▼ Index [0]	3 members
Monster ID	1001
Count	2
Level	1
▼ Index [1]	3 members
Monster ID	1001
Count	1
Level	1



```
void AStageSection::BeginSection()  
{  
    State = EState::IN_PROGRESS;  
    SpawnMonster();  
}
```

```
void AStageSection::SpawnMonster()  
{  
    // 게임모드에게 몬스터 스폰 요청  
    TObjectPtr<ACombatGameMode> GameMode = GetWorld()->GetAuthGameMode<ACombatGameMode>();  
    if (nullptr == GameMode)  
    {  
        State = EState::CLEARED;  
        return;  
    }  
  
    SpawnedCount = GameMode->SpawnMonsterOnSection(Index, SpawnPoints);  
  
    if (SpawnedCount > 0)  
        EventHandle = GameMode->StageEvent[EStageEvent::HUNT].AddUObject(this, &AStageSection::OnMonsterDead);  
    else  
        State = EState::CLEARED;  
}
```

```
void ACombatGameMode::RegisterObjectPool()  
{  
    UObjectPoolManager* ObjectPool = GetWorld()->GetSubsystem<UObjectPoolManager>();  
    UDataManager* DataManager = GetGameInstance()->GetSubsystem<UDataManager>();  
  
    // 특정 구역에서 요구하는 최대 개수 구하기  
    ...  
  
    // 오브젝트 풀링 등록  
    for (const TPair<EMonsterType, uint8>& pair : MaxCountPerType)  
    {  
        EMonsterType Type = pair.Key;  
        TFunction<TObjectPtr<AActor>()> CreateFunc =  
            [this, Type]()  
            {  
                // 몬스터 객체 생성 원리식  
                FActorSpawnParameters SpawnParam;  
                SpawnParam.Owner = this;  
                SpawnParam.SpawnCollisionHandlingOverride = ESpawnActorCollisionHandlingMethod::AlwaysSpawn;  
  
                TObjectPtr<AMonsterBase> Inst = GetWorld()->SpawnActor<AMonsterBase>(MonsterClass[Type], SpawnParam);  
                check(Inst != nullptr);  
  
                // 몬스터 사망 시, 오브젝트 풀로 복귀하도록 이벤트에 배인딩  
                Inst->OnMonsterDead.BindUObject(this, &ACombatGameMode::ReleaseMonster);  
  
                return Inst;  
            };  
  
        ObjectPool->Register(MonsterClass[pair.Key], CreateFunc, pair.Value);  
    }  
    ...  
}
```

핵심 기능 3. 스테이지

이벤트 기반 진행

스테이지 진행은 발행-구독 모델을 확장한 이벤트 버스 방식으로 구조를 잡았습니다.
게임 진행 간에 개별 액터들이 진행 과정에 직접적으로 관여하지 않도록
GameMode를 통한 이벤트 기반으로 작업했습니다.

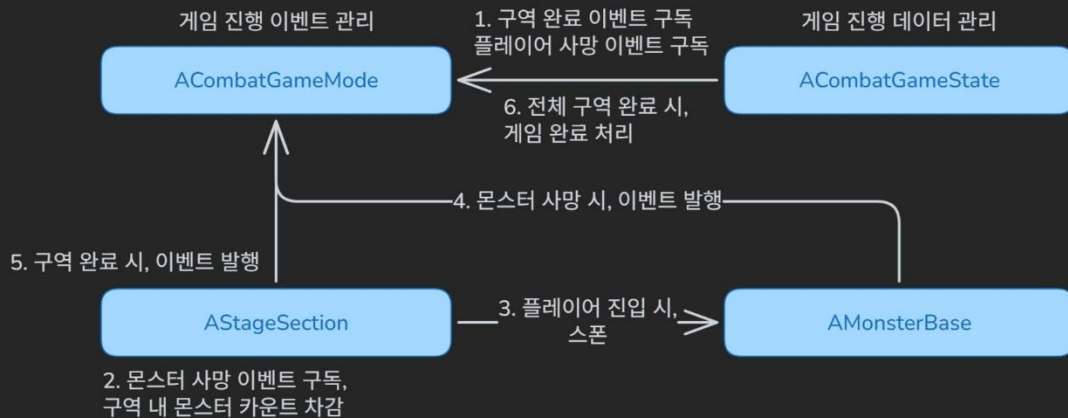
스테이지 진행에 발생하는 이벤트 종류

```
enum class EStageEvent : uint8
{
    HUNT,           // 몬스터 사망
    SECTION_CLEAR,  // 구역 완료
    PLAYER_DEAD,    // 플레이어 사망

    END
};
```

구역 완료 정보 같이 진행에 필요한 주된 데이터는 GameState에서 관리합니다.
StageSection은 몬스터 사망 이벤트를 구독해두고, 스폰된 몬스터가 죽으면 몬스터 카운트를 낮춥니다.
몬스터 카운트가 0이되면 해당 StageSection은 구역 완료 이벤트를 발행합니다.

구역 완료 이벤트가 발행되면 GameState에서 해당 구역을 완료처리하고
모든 구역이 완료되면 GameMode에서 게임을 완료처리 합니다.



```
void AMonsterBase::OnDead()
{
    // 사망 시 처리
    ...
    // 몬스터 사망 이벤트 호출
    ACombatGameMode* GameMode = GetWorld()->GetAuthGameMode<ACombatGameMode>();
    GameMode->StageEvent[EStageEvent::HUNT].Broadcast({ GetSectionID(), this });
    ...
}
```

```
void AStageSection::OnMonsterDead(const FStageEventContext& _context)
{
    if (_context.SectionIndex != Index)
        return;

    check(SpawnedCount > 0);
    SpawnedCount--;

    if (SpawnedCount == 0)
        EndSection();
}
```

```
void AStageSection::EndSection()
{
    State = EState::CLEARED;

    TSharedPtr<ACombatGameMode> GameMode = GetWorld()->GetAuthGameMode<ACombatGameMode>();
    GameMode->StageEvent[EStageEvent::HUNT].Remove(EventHandle);

    FStageEventContext Context;
    Context.SectionIndex = Index;
    GameMode->PublishEvent(EStageEvent::SECTION_CLEAR, Context);
}
```

```
void ACombatGameState::Init(uint8 _playerCnt, const TArray<FSection>& _section)
{
    PlayerCount = _playerCnt;
    bSectionCleared.SetNum(_section.Num());

    for (uint8 i = 0; i < _section.Num(); i++)
        bSectionCleared[i] = _section[i].Spawn.Num() == 0;

    ACombatGameMode* GameMode = GetWorld()->GetAuthGameMode<ACombatGameMode>();
    GameMode->StageEvent[EStageEvent::PLAYER_DEAD].AddLambda(
        [this](const FStageEventContext& _context)
        {
            OnPlayerDead.ExecuteIfBound(--PlayerCount);
        }
    );

    GameMode->StageEvent[EStageEvent::SECTION_CLEAR].AddLambda(
        [this](const FStageEventContext& _context)
        {
            bSectionCleared[_context.SectionIndex] = true;
            OnSectionCleared.ExecuteIfBound(GameIsCleared());
        }
    );
}
```

아이템

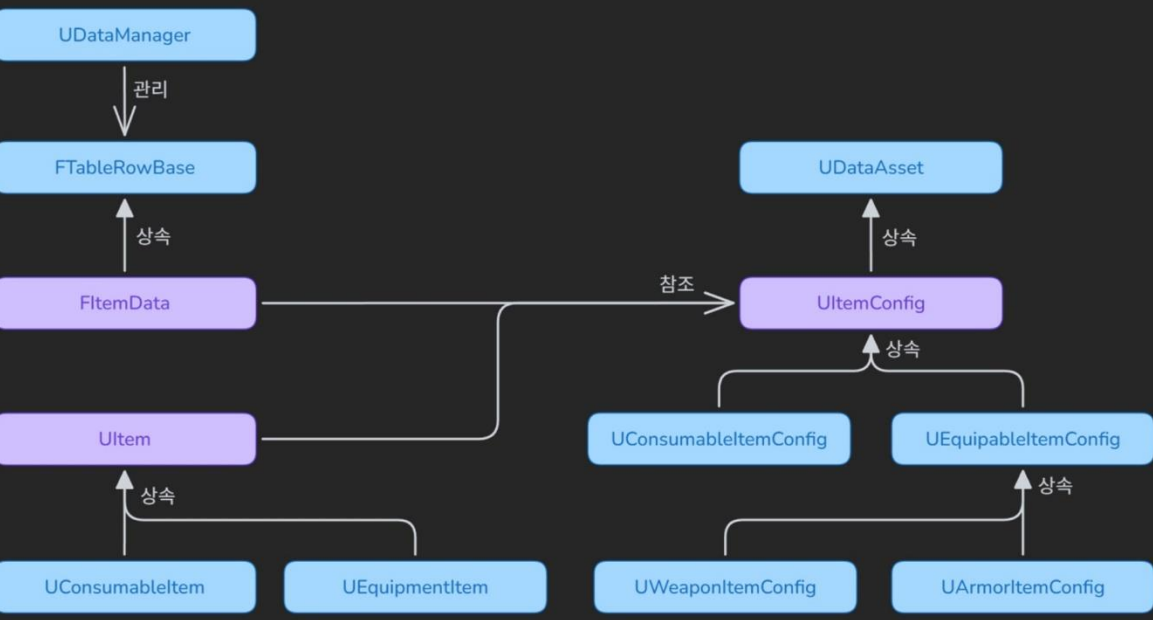
아이템 구조

- 플레이어 캐릭터가 사용하는 아이템의 종류는 크게 3가지로 나눌 수 있었습니다.
- 일반 아이템 : 재료로 사용하여 아이템 제작에 사용합니다.
 - 소비 아이템 : 아이템을 소비하여 플레이어에게 각종 효과(버프, 디버프)를 제공합니다.
 - 장비 아이템 : 장착 시, 플레이어의 스탯을 상승시킵니다.

아이템 데이터

데이터테이블과 데이터 에셋을 사용하여 다양하고 많은 아이템의 데이터를 추가하기 쉽게 작업했습니다.

데이터테이블을 사용해서 대량의 아이템 데이터를 관리하고
종류마다 세부적인 정보는 데이터 에셋으로 모듈화하는 방향으로 작업했습니다.



아이템 ID 기반 사용

```
void ACombatGameMode::GameClear()
{
    // 보상 지급
    // 골드 재화 지급
    ...
    // 아이템 지급
    for (const FRewardItem& Item : StageData->RewardItems)
        PlayerManager->AddItem(Item.ID, Item.Count);

    OnGameEnd.Broadcast(true, StageData);
}

uint8 UPlayerManager::AddItem(const FName& _itemID, int32 _amount)
{
    //아이템 추가
    UInventory::FCreateItemParam Param(_itemID, _amount);
    Inventory->TryAddItem(Param);

    return Param.OutIndex;
}
```

Row	Type	Item
1	1001 Item	/Script/ARPG_Hunter.ItemConfig/Game/03-Data/ItemConfig/1001
2	1002 Item	/Script/ARPG_Hunter.ItemConfig/Game/03-Data/ItemConfig/1002
3	1003 Item	/Script/ARPG_Hunter.ItemConfig/Game/03-Data/ItemConfig/1003
4	1004 Item	/Script/ARPG_Hunter.ItemConfig/Game/03-Data/ItemConfig/1004
5	2001 Consumable Item	/Script/ARPG_Hunter.ConsumableItemConfig/Game/03-Data/Item
6	2002 Consumable Item	/Script/ARPG_Hunter.ConsumableItemConfig/Game/03-Data/Item
7	2003 Consumable Item	/Script/ARPG_Hunter.ConsumableItemConfig/Game/03-Data/Item
8	2004 Consumable Item	/Script/ARPG_Hunter.ConsumableItemConfig/Game/03-Data/Item
9	3001 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
10	3002 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
11	3003 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
12	3004 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
13	3005 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
14	3006 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
15	3007 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
16	3008 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC
17	3009 Armor Item	/Script/ARPG_Hunter.EquipmentItemConfig/Game/03-Data/ItemC

Row Editor

3001

Type: Armor Item

Item: 3001_BasicHelmet

장비 아이템 데이터 에셋

소비 아이템 데이터 에셋

랜드 스케이프 적용 후 에러

문제 발생

플레이어와 몬스터 기반 기능을 작업하고 나서 랜드 스케이프를 이용해 레벨을 만들어 테스트하던 도중 에러 발생했습니다.

원인 분석

최초 몬스터 액터를 스폰 시, nullptr이 반환되어 에러가 발생했습니다.

기존 테스트에 쓰던 레벨에선 발생하지 않는 것을 확인했고 새로 작업 중인 레벨과의 차이점은 랜드 스케이프 사용 유무임을 확인했습니다.

랜드 스케이프에서 액터 스폰 시, 일반 스택틱 메쉬와 달리 Z축 높이와 충돌 문제가 있음을 확인했습니다.

기존에 쓰던 평평한 메시에서와 달리 Z축 높이 차이와 랜드 스케이프와 충돌로 인해서, 스폰이 실패 처리되어 nullptr이 반환되었습니다.

해결 방법

액터 스폰 시, FActorSpawnParameters에 SpawnCollisionHandlingOverride 설정을 AlwaysSpawn을 변경 후 해결했습니다.

```
// 오브젝트 풀링 등록
for (const TPair<EMonsterType, uint8>& pair : MaxCountPerType)
{
    EMonsterType Type = pair.Key;
    ObjectPool->Register(
        MonsterClass[pair.Key],
        [this, Type]()
        {
            // 몬스터 액터 생성 함수식
            FActorSpawnParameters SpawnParam;
            SpawnParam.Owner = this;
            TObjectPtr<AMonsterBase> Inst = GetWorld()->SpawnActor<AMonsterBase>(MonsterClass[Type], SpawnParam);
            check(Inst != nullptr); X

            // 몬스터 사망 시, 오브젝트 풀로 복귀하도록 이벤트에 바인딩
            Inst->OnMonsterDead.BindUObject(this, &ACombatGameMode::ReleaseMonster);

            return Inst;
        },
        pair.Value);
}
```

```
LogWorld: Bringing World /Game/01-Level/Dungeon/UEDPIE_0_Stage1.Stage1 up for play (max tick rate 0) at 2026.03.24-22.55.08
LogWorld: Bringing up level for play took: 0.100935
LogSpawn: Warning: SpawnActor failed because of collision at the spawn location [X=0.000 Y=0.000 Z=0.000] for [BP_RegularMonster_C]
LogOutputDevice: Warning:
```

```
for (const TPair<EMonsterType, uint8>& pair : MaxCountPerType)
{
    EMonsterType Type = pair.Key;
    TFunction<TObjectPtr<AActor>(>) CreateFunc =
        [this, Type]()
        {
            // 몬스터 액터 생성 함수식
            FActorSpawnParameters SpawnParam;
            SpawnParam.Owner = this;
            SpawnParam.SpawnCollisionHandlingOverride = ESpawnActorCollisionHandlingMethod::AlwaysSpawn;

            TObjectPtr<AMonsterBase> Inst = GetWorld()->SpawnActor<AMonsterBase>(MonsterClass[Type], SpawnParam);
            check(Inst != nullptr);

            // 몬스터 사망 시, 오브젝트 풀로 복귀하도록 이벤트에 바인딩
            Inst->OnMonsterDead.BindUObject(this, &ACombatGameMode::ReleaseMonster);

            return Inst;
        };

    ObjectPool->Register(
        MonsterClass[pair.Key],
        CreateFunc,
        pair.Value);
}
```

레벨 시퀀스 적용 후 빌드 에러

문제 발생과 원인

보스 몬스터의 등장 연출을 위해 레벨 시퀀스를 이용한 컷씬 작업 후 발생했습니다.
에디터를 통한 테스트 이후, 빌드 테스트를 진행하다 에러를 확인했습니다.

에러 로그를 살펴보니 DataManager 초기화 중 스테이지 데이터테이블을 참조하는 과정에서 문제가 발생했습니다.
스테이지 데이터테이블에서 레벨 시퀀스를 참조하고 있고 레벨 시퀀스에서 사용하는 특정 액터가 아직 등록되지 않아 문제가 발생했습니다.

```
USTRUCT(BlueprintType)
struct ARPG_HUNTER_API FStageData : public FTableRowBase
{
    GENERATED_BODY()
public:
    ...
    // 스테이지에서 재생할 컷씬 LevelSequence
    UPROPERTY(EditAnywhere)
    TArray<TObjectPtr<class ULevelSequence>> CutSceneAssets;
};
```

Call Stack

Search (Ctrl+E) 🔍 ⏪ ⏩ 🧵 View all Threads 📄 Show External Code

Name
UnrealEditor-ARPG_Hunter-Win64-DebugGame.dll!ConstructorHelpersInternal::FindOrLoadObject<UDataTable> (FString & PathName, unsigned int LoadFlags) Line 39
UnrealEditor-ARPG_Hunter-Win64-DebugGame.dll!ConstructorHelpers::FObjectFinder<UDataTable>::FObjectFinder<UDataTable> (const wchar_t * ObjectToFind, unsigned int InLoadFlags) Line 90
➡ UnrealEditor-ARPG_Hunter-Win64-DebugGame.dll!UDataManager::UDataManager() Line 46
UnrealEditor-ARPG_Hunter-Win64-DebugGame.dll!InternalConstructor<UDataManager> (const FObjectInitializer & X) Line 3750

[[2026.03.24-14.05.42:348][0]]LogStreaming: Warning: LoadPackage: SkipPackage: /Game/ParagonMurdock/FX/Materials/ParamCollections/GlobalArtCollection (0x3FF8A7D396ED3080) - The package to load does not exist on disk or in the loader
[[2026.03.24-14.05.42:349][0]]LogUObjectGlobals: Warning: Failed to find object 'Object /Game/ParagonMurdock/FX/Materials/ParamCollections/GlobalArtCollection/GlobalArtCollection'
Ensure condition failed: Registry [File:D:\build\++UE5\Sync\Engine\Source\Runtime\Engine\Private\Elements\Framework\EngineElementsLibrary.cpp] [Line: 35]
Typed element was requested for '/Game/01-Level/Sequence/LS_1003.LS_1003:MovieScene_0.BP_5' before the registry was available! This usually means that NewObject was used instead of CreateDefaultSubobject during CDO construction.
[[2026.03.24-14.05.42:466][0]]LogOutputDevice: Error: Ensure condition failed: Registry [File:D:\build\++UE5\Sync\Engine\Source\Runtime\Engine\Private\Elements\Framework\EngineElementsLibrary.cpp] [Line: 35]
Typed element was requested for '/Game/01-Level/Sequence/LS_1003.LS_1003:MovieScene_0.BP_5' before the registry was available! This usually means that NewObject was used instead of CreateDefaultSubobject during CDO construction.

레벨 시퀀스 적용 후 빌드 에러

해결 방법

등록되지 않았다는 로그를 보고 빌드 시 에셋 누락이 발생했다고 생각했습니다.
프로젝트 세팅에서 패키징 시, 특정 에셋을 반드시 포함하도록 설정하고 다시 빌드를 시도했습니다.
그럼에도 빌드는 실패했고 동일한 에러 로그가 출력됐습니다.

에셋 누락이 아니라면 데이터테이블에서 레벨 시퀀스를 직접 참조하기 때문에 발생한 것이라 판단했고
검증을 위해서 데이터테이블에서 레벨 시퀀스를 TObjectPtr이 아닌 TSoftObjectPtr로 변경했습니다. (하드 레퍼런스 -> 소프트 레퍼런스로 변경)
그 후 다시 빌드 시도에서 성공한 것을 확인했습니다.

레벨 시퀀스를 사용하는 부분에서 레벨 시퀀스 에셋을 비동기 로드하도록 변경 후 사용하는 것으로 문제를 해결했습니다.

```
USTRUCT(BlueprintType)
struct ARPG_HUNTER_API FStageData : public FTableRowBase
{
    GENERATED_BODY()
public:
    ...
    // 스테이지에서 재생할 컷씬 LevelSequence

    UPROPERTY(EditAnywhere)
    TArray<TSoftObjectPtr<class ULevelSequence>> CutSceneAssets;
};
```

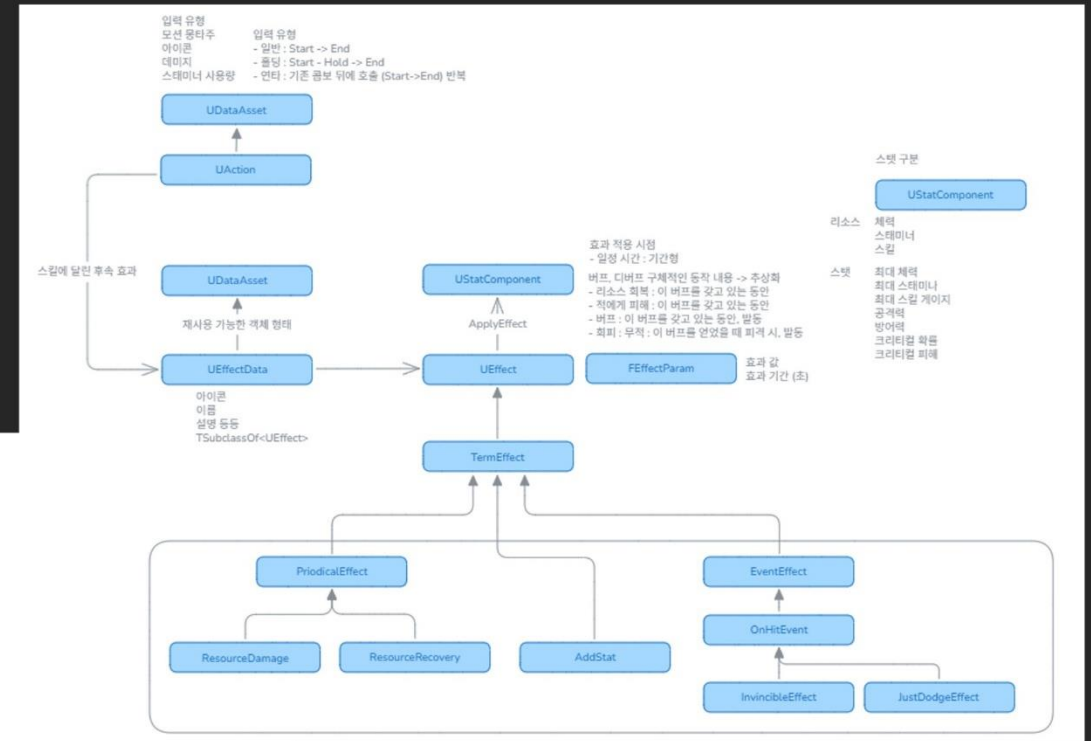
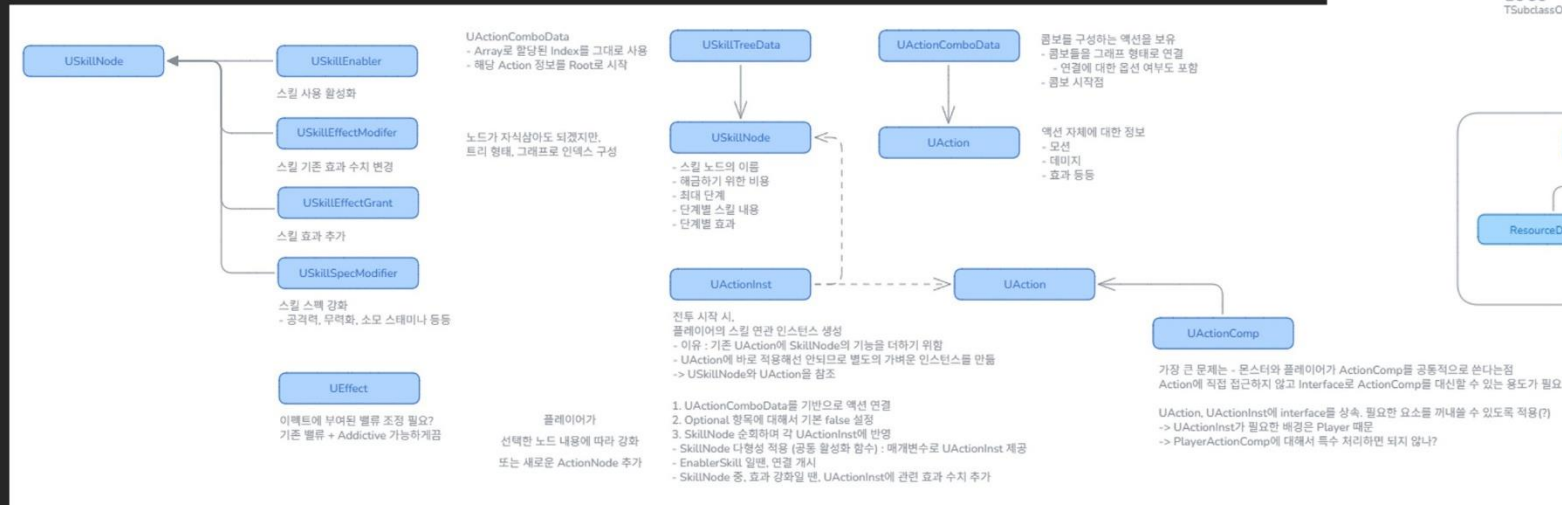
```
void AStageCutSceneSection::BeginPlay()
{
    Super::BeginPlay();

    ...
    CutSceneAssetPath = CutSceneAssetData->LevelSequence;

    FStreamableManager& Streamable = UAssetManager::GetStreamableManager();
    Streamable.RequestAsyncLoad(
        CutSceneAssetPath.ToSoftObjectPath(),
        FStreamableDelegate::CreateUObject(this, &AStageCutSceneSection::OnCutSceneAssetLoaded)
    );
}
```

비록 프로젝트 중반에 GAS에 대해 알게 되어 아쉬움이 있었지만
밑바닥부터 GAS와 유사한 구조 설계하는 과정은 오히려 GAS의 구조 대해 이해하는데 큰 도움이 되었습니다.
유사한 구조를 만들어보며 고민하고 구현해보았기에
GAS의 Ability, AttributeSet, Effect와 같은 요소들이 분리된 이유와 의도에 대해 이해할 수 있었습니다.

프로젝트 중 구조 설계 메모 : 스킬 및 액션



프로젝트 중 구조 설계 메모 : 이펙트 구조